

The Agent Experience Practitioner Playbook

How to evaluate, diagnose, and improve your technology's agent experience

Part 1: Why AX matters

Chapter 1. The game has changed

Whether you build a technology or help developers adopt it, your audience now includes AI coding agents. Agents pick technology stacks and choose services. Left alone, they make these choices based on stale training data and whatever extensions happen to be loaded. You need to influence agents now, not just developers.

Ask a coding agent to build something with the technology you work on: a Cosmos DB app, a SharePoint Framework (SPFx) web part, a payments integration, whatever is in your space, and watch what happens.

The agent will generate code. It will look reasonable and might even compile. But look closer and you'll find the wrong SDK version, a deprecated authentication pattern, a partition key strategy that no one on the product team would recommend. The agent didn't make a random mistake. It did exactly what its training data told it to do, and its training data is months or years behind what the product team shipped last quarter.

How close does the agent get to correct usage? That's the agent experience of your technology. Measuring it with evals is gaining popularity, but most teams are still figuring out how to do it well. Many haven't started yet and judge how their extensions perform on anecdotal observations.

Agents are another audience

Product teams, maintainers, and developer advocates have always helped developers succeed with technology. You build products and extensions. You write docs and samples. You make the technology approachable, and you make sure developers can find what they need when they need it.

All of that still matters. But it was built for a world where the developer reads your docs, studies your samples, and writes their own code. Developers increasingly work through AI coding agents instead. They describe what they want, and the agent figures out how to build it. The agent picks which SDK to use, which version to install, which authentication pattern to follow, which configuration to apply. The developer reviews the result, but the agent made the decisions.

When agents make those decisions, they draw on their training data, tool descriptions, whatever extensions are loaded, and whatever docs they can reach. If your technology isn't represented well in those sources, agents will get it wrong. Not sometimes. Reliably. Across every developer who uses that agent with your technology. Albeit to different degrees and in different ways.

That changes what product and advocacy work means. It isn't enough that a developer can find the right information. An agent needs to find and correctly apply it through the surfaces you control: docs, MCP tools, skills, plugins, instructions, CLIs, and APIs.

Newer models don't necessarily help either. A knowledge cutoff tells you the latest date a model's training data could cover, not how well the model knows your product. Its product knowledge might end months earlier or be patchy across the product's history, and we've seen a newer model know *less* about a technology than its predecessor. [Knowledge cutoff is a poor proxy for model capability](#), so test the baseline to understand what you're up against.

Waiting for models to get better isn't a strategy. Model improvements are unpredictable, and developers are using your technology today. Fixing the sources agents rely on improves the current experience and gives future models better information to work with.

What this looks like in practice

In fall 2025, we in DevRel at Microsoft started measuring how agents actually perform with the technologies we support. We've since run hundreds of agent sessions across Azure, Cosmos DB, SPFx, and Microsoft 365 Copilot extensions, using the same kinds of prompts real developers use. What we found wasn't subtle.

Agents frequently ignored available CLI tools and manually edited files, introducing version errors the CLI would have prevented. They picked up documentation but anchored on the wrong section, collapsing three distinct architectural patterns into one because a single docs page overemphasized it. They loaded extension tools but never called them, choosing to rely on training data that was two SDK versions behind.

These aren't edge cases. They're patterns that repeat across technologies and agent harnesses. And they repeat because few teams are looking.

What happened when we looked

Looking was the easy part. What mattered was what came after: turning findings into specific changes the owning teams could ship. Across the technologies we evaluated, that turned into dozens of shipped fixes to docs and agent extensions. For the [Azure Cosmos DB Agent Kit](#) alone, you can browse the [46 improvements that came out of our evaluations](#). The [SPFx dev skill](#) followed the same path.

One example: SPFx release notes used filenames like `release-1.22.md`. That seems fine to a human, but an agent looking for version 1.22.2 sees `release-1.22` and assumes it has what it needs. It never looks for the version-specific page. Renaming the file to `release-1.22.0.md` made it obvious that the page covered 1.22.0 and nothing later, so the agent realized there was a gap and went looking for the right version. Before the change, none of

the 4 runs that read the release notes went on to the 1.22.2 page. After it shipped, 4 of 5 runs did.

Another: SPFx docs included a tip pointing developers to the [CLI for Microsoft 365](#) with the exact upgrade command. No run used the CLI, because the agent had already formed a plan from training data. Tips are suggestions, and agents treat them that way. After we replaced the tip with a warning that said "manually updating `package.json` alone will result in build failures" and rewrote the migration guide, every run used the CLI instead. The agent didn't need encouragement. It needed its existing plan invalidated.

These are the kinds of improvements that compound. Each one is small. Together they transform how reliably agents work with a technology.

Who this is for

This playbook is for anyone who builds technology or extensions that AI coding agents use. If you build an SDK, API, service, CLI, MCP server, skill, plugin, or instruction set, or write the docs behind them, you control what agents discover and apply. That includes product teams and open-source maintainers, docs writers, and platform teams building internal tooling for their own developers. Evaluation shows you whether those surfaces work as intended.

If you advocate for a technology rather than own it, you bring the scenario knowledge and sustained attention needed to evaluate the developer experience across those surfaces. You can run the work with the owning team or bring evidence that helps them decide what to change. Chapter 11 covers what to do when you don't have a direct line to the owner.

Either way, the work depends on domain expertise. Without it, criteria end up miscalibrated: too easy and the base model passes already, telling you nothing. Too hard and you optimize for something theoretical that no developer actually needs. The strongest evaluations combine product knowledge with realistic developer scenarios, whether that's one person or two teams working together. If you're looking to tune your own coding agent setup, some of the ideas here transfer, but that's not what this playbook is about.

What you'll be able to do

By the end of this playbook, you'll be able to:

1. **Evaluate your technology.** Run an AX evaluation against the products, extensions, SDKs, and APIs you build or support, and understand what the results mean.
2. **Diagnose failures.** Identify whether agents fail to discover your technology or apply it correctly, and trace root causes to specific docs, tools, or extension surfaces.
3. **Drive improvements.** Translate findings into specific changes, test them, and work with the people who own each surface to ship fixes.

Further reading

- [How to improve AI coding agent support for your SDK, API, or CLI.](#) The agent stack layer by layer: what's fixed, what you control, and where you can win.

Part 2: How to evaluate AX

Chapter 2. The evaluation model

Benchmarks like SWE-bench measure average model capability across a standard task set. They're useful for comparing models against each other, but they can't tell you whether an agent uses *your* technology correctly. That requires your domain knowledge. You know the right SDK version and the authentication pattern the product team recommends. A benchmark doesn't have that information and can't check for it.

AX evaluation answers a more useful question. Given your technology, does the agent use it the way it should? That question is only worth asking if you can trust the answer, and an evaluation can lie to you convincingly. We've seen evaluations report perfect scores for code that never compiled, because nobody attempted to build it. We've seen a "used Platform X"-check pass whether or not the agent used Platform X, because it matched the word and never checked the meaning. A result that looks confident and turns out to be wrong is worse than no result, because you'll act on it.

The model in this chapter is the set of commitments that make a result trustworthy. Whatever system you run it in, a serious AX evaluation holds to the same discipline.

Every evaluation is built from four things. **Scenarios** are the prompts developers type into coding agents, run against a workspace: blank or pre-populated. **Criteria** are the checks that decide whether the agent made the right choices. **Profiles** are the environments you run in: operating system, harness, model and its configuration, and whatever extensions are loaded. **Gates** are the mechanical checks that confirm the generated code works. Each of these has a standard to meet before you can trust what it tells you.

Criteria judge meaning, and judge it the same way twice

Criteria are where your domain expertise becomes measurable. You know the correct SDK version and the partition key strategy the product team would sign off on. A criterion captures one of those judgments so an LLM judge can rule on it: pass, fail, or skip.

Two things make a criterion trustworthy. The first is that it checks meaning. "The response mentions Platform X" passes whether the agent embraced Platform X or argued against it, so matching words hands you a confident wrong answer. A real criterion reads the generated artifacts in context and judges what the agent actually did. The second is that it reaches the same verdict every time. If the same output scores pass on one run and fail on the next, the criterion is ambiguous, and the problem is in your wording. Chapter 5 is entirely about writing good criteria.

To further understand the agent's strengths and weaknesses, you can group criteria into areas, such as the code following recommended patterns or dependencies being up to date. It's helpful, but you're not required to do it. Each criterion just needs to test something that matters to you and rule on it consistently.

Run in the environment your developers use

A result only means something if it comes from the environment your developers work in. The operating system, harness, model, and extensions in context all change how the agent behaves, and the harness matters more than people expect. Two surfaces from the same vendor can run different harnesses, so a strong result in one doesn't carry over to another, even on the same model. Running a CLI on Linux because it's easy to automate and cheap to run, when your developers work in an IDE on Windows, gives you a precise number about a situation none of them are in. You'll fix problems they don't have and miss the ones they do. Pick the profile that represents most of your audience and make that your reference

point. Chapter 6 goes deeper on choosing profiles, and Chapter 7 on reproducing them faithfully.

Prove the code runs, don't assume it

Gates are mechanical, deterministic checks, and they stack, so each one runs only if the previous one passed. A **prerequisite gate** comes first. Did the agent pick the right technology? If you asked for Cosmos DB and it built on MongoDB, nothing else is worth measuring. After that, the gates follow the code's own lifecycle: does it **build**, do **tests** pass (if any), does the app **run**, and does it **deploy** when deployment is part of the job. Deploy comes last because there's no sense deploying something you haven't confirmed runs.

Skip these and you get the most common lie in agent evaluation: a perfect criteria score for a project that doesn't compile. Gates are what stop that from reaching your readout.

Compare against a baseline, and count the cost

A score on its own tells you very little. What you want to know is whether your extension made things better or worse, so every result is measured against a **bare baseline**: the same scenario and profile, with no extensions. Where the extension helps, you have **lift**, and where it hurts, **drag**.

Keep the score easy to understand. Take the pass rate with your extension, subtract the pass rate of the baseline, and you have an answer a product owner can read and act on without a data science degree. If you grouped criteria into areas, do this per area and compare across them.

Be mindful of two traps. The first is treating usage as success. The agent calling your extension doesn't mean it helped. If the numbers don't move, the extension is token cost without benefit. The second is ignoring what the task costs to run. Most developers aren't on unlimited token plans. A profile that lifts quality but at three times

the price forces a real cost-benefit decision, and you'll miss that conversation if you only focus on an abstract score. Measure cost alongside quality, every time.

Run it more than once

Language models are non-deterministic. Send the same prompt five times and you can get five different results. A single run tells you almost nothing, because it might be the outlier. Run each scenario and profile combination several times, at least five, so you can tell a real pattern from noise. Five runs won't give you statistical significance but, in our experience, it's a practical starting point: enough for results to cluster and obvious failures to stand out, without making evaluations too expensive to run regularly. If results are mixed, add more runs. A criterion that fails once in five is variance. But a criterion that fails five times in five is a problem you can fix.

Keep the evidence

A score tells you what happened. It can't however tell you why. For that you need the full record of the run: every tool the agent called, every file it read, every decision it made along the way. Keep that trajectory for every run, along with the generated output and the verdicts. Without it, you can see that a criterion failed but you're guessing at the cause, and guessing is how you end up fixing the wrong thing. Chapters 8 and 9 are about reading this evidence.

What this gives you

Hold to these commitments and you get a result you can defend: a clear picture of what the agent does with your technology, an honest comparison of whether your extensions help or hurt and at what cost, and specific failures that point to specific fixes. You can point past a headline number like “the agent scored 72%” to the cause behind it: the agent installs the wrong SDK in 4 out of 5 runs because a docs page hardcodes an outdated version.

Further reading

- [What AI benchmarks are not telling you](#). Why leaderboard scores don't transfer to your technology, and how to start a small, representative eval of your own.
- [How to measure AI agent extension effectiveness](#). Why you can't judge an extension by looking at it, and what measuring lift, drag, and cost actually involves.

Chapter 3. The eval lifecycle

You already know what good looks like for your technology. The eval lifecycle is how you make that knowledge visible in data that can drive decisions and improvements.

The process has five steps:

1. **Design a scenario.** Write the prompt a developer would give a coding agent.
2. **Write evaluation criteria.** Define what “correct” looks like for the output. Writing criteria isn’t trivial and we discuss it in more detail in chapter 5.
3. **Choose agent profiles.** Pick which combinations of harness, model, and extensions to run against.
4. **Run the evaluation.** Execute each scenario-profile combination multiple times and collect the outputs and trajectories.
5. **Generate and interpret the readout.** Turn the raw results into an actionable report.

After step 5, you have data. What you do with that data (investigating trajectories and driving improvements) is Part 3.

The chapters that follow cover each step in detail. If a step feels mechanical, it’s short. If it requires judgment, it’s longer. Chapter 7 also covers what to look for in an evaluation system.

Chapter 4. Designing scenarios

A scenario isn't a test case. It's the question you're asking about your technology. The quality of what you learn depends entirely on the quality of the question you ask.

A scenario is the prompt a developer would type into a coding agent. Nothing more. No evaluation instructions mixed in, no scoring rubric, no hints about what the "right" answer is. The agent should see exactly what a real developer would type. Add anything more, and the results will be skewed.

What makes a good scenario

The only true requirement: your scenario should be as close to what a developer would actually put into a coding agent as possible. If developers use vague or ambiguous prompts, that's exactly what you should be testing. The moment you start crafting artificially clear (or ambiguous) prompts, you're losing touch with reality.

A scenario can be anything from "build me a todo app with Cosmos DB" to "upgrade this project to the latest version" (yes, explicitly leaving out the tech stack to see if the agent properly discovers it!). Both are valid because both are things a developer would ask in The Real World™.

Propensity vs. efficacy

You can test two different things, and both matter for different reasons. Which you start with depends on your business context.

Propensity (discovery): does the agent find your technology and choose it? "Build an API that stores customer data in a document database" doesn't name Cosmos DB. If the agent picks MongoDB instead, that's a propensity finding. If a product is new, propensity is probably where you start. You need to know whether agents even

acknowledge your product, and if they don't, what other technologies are competing for attention.

Efficacy (quality): when the agent uses your technology, does it use it correctly? "Create a Cosmos DB container with hierarchical partition keys for multi-tenant isolation" clearly steers the technology choice. Now you're testing whether the agent knows the right patterns. If you're working on technology that's existed for years and models already know about it, efficacy is where you'll find the most value, unless you're looking at competitive scenarios where propensity matters again.

Neither is better than the other. Start with the one most relevant to your situation.

Things to be aware of

Complex prompts increase variance. A prompt that asks the agent to do five things will produce more variable results than a simple one. That's fine but be aware that you'll need more precise evaluation criteria to interpret what happened. The more complex the prompt, the more careful you need to be about what you're checking in the evaluation.

Prompt-crafting is a trap. If your eval only works with perfectly worded prompts, it won't tell you what happens in the real world. Developers don't use your team's internal vocabulary. They don't always know the right product names. Be mindful of that gap.

Your turn

Pick a scenario for your technology. Before you run it, check:

- Is it something a real developer would *actually* type into a coding agent?
- Are you aware of what you're testing (propensity or efficacy)?
- If the agent gets it wrong, will your evaluation criteria be able to surface *what* went wrong?

The quality checklist in Appendix A covers this in more detail.

Further reading

- [Models don't have preferences, they have context](#). Why "the model prefers X" from a bare chat prompt tells you little, and why propensity tests need a realistic workspace and real tasks.

Chapter 5. Writing evaluation criteria

Writing criteria is where your domain expertise becomes measurable.

This is the hardest part of the process. Get it right and your eval produces actionable signal. Get it wrong and you'll get scores that look confident, feel consistent, and mean nothing.

What criteria are

Criteria are the rules your evaluation system uses to judge whether the agent made the right *choices*: the right SDK version, authentication approach, or product pattern. An LLM judge reads the generated artifacts and decides whether each criterion passes, fails, or should be skipped. Build and run behavior belongs in mechanical gates instead.

ASIDE: What about deterministic judges? There are pros and cons to using deterministic judges or LLM judges when doing coding agent evals. Deterministic judges can run faster and return more predictable results, but they are often more brittle and prone to evaluation errors caused by incorrect pattern matching. LLM judges, meanwhile, are slower and require careful configuration to produce repeatable, usable results, but they are better at evaluating the code in context and avoiding simple pattern match errors. A mix is often useful, but it is important to be aware of the limits and advantages of both.

What good criteria look like

Good criteria test things that matter to the product and its users and express each check precisely enough that the judge reaches the same verdict each time. "The code uses Cosmos DB" works as a prerequisite gate, but it doesn't tell you whether the agent used Cosmos DB well.

The judge is an LLM too, with its own knowledge gaps. "Uses the current SDK" forces it to decide what current means. Name the package, accepted version range, APIs, and specific wrong patterns instead.

For Cosmos DB, meaningful criteria might check:

- Does it use hierarchical partition keys where the scenario calls for multi-tenant isolation? (Specific: "Uses PartitionKeyBuilder or equivalent hierarchical partition key API. Fails: uses a single flat partition key like /customerId")
- Does it use identity-based authentication? (Specific: "Passes: code uses DefaultAzureCredential, ManagedIdentityCredential, or equivalent. Fails: connection string contains AccountKey=")

For an SPFx upgrade, name the exact target version and enumerate the required files and properties, such as dependency versions in package.json and the generator version in .yo-rc.json. Put each required change in a separate criterion so you can see exactly what the agent missed.

Writing criteria that work

Each criterion needs four things:

1. **What you're checking.** One specific aspect of the output.
2. **What passes.** The concrete conditions that indicate correct usage. Name packages, versions, APIs.
3. **What fails.** The concrete conditions that indicate incorrect usage. Name the specific wrong patterns.
4. **When to skip.** When the criterion doesn't apply to the output.

Criteria rarely land right on the first attempt. Here's how one evolves:

Draft 1: "Uses current SDK version." *Current* means nothing to the judge, so the criteria must supply its own version.

Draft 2: "Uses Azure Cosmos DB SDK v4 for JavaScript (@azure/cosmos)." Better, but still ambiguous. Does any v4 release count? What about a prerelease?

Draft 3: "Uses @azure/cosmos v4.x. Passes: package.json specifies a stable version from 4.0.0 up to, but not including, 5.0.0. Fails: it specifies an earlier major version, a prerelease version, or the deprecated documentdb package. Skipped: the application doesn't use Cosmos DB." Now the judge doesn't need to define *current* or decide whether a prerelease counts.

You can also use a model to help with this. Ask it whether your criteria are ambiguous and how to clarify them. Models are good at catching ambiguity, and it'll save you time running and re-running evaluations to discover the issue.

Calibrate before you lock

A criterion can read clearly and still produce the wrong verdict or score inconsistently. Before using a criteria set for a production evaluation, run a calibration cycle:

1. Test each criterion against an artifact you know should pass and one you know should fail.
2. Score the same artifacts repeatedly to see whether the judge reaches the same verdict.
3. Have someone who knows the product confirm technical accuracy.
4. Confirm that the wording produces consistent verdicts, using the repeated scores from step 2.

5. Revise and repeat until the criteria are both accurate and consistent.

Calibration runs are for improving the criteria, not measuring AX. Keep them separate from production results. Once the criteria set is locked, don't rewrite it in response to a surprising score. Investigate the output and trajectory first. The result might be telling you something real.

Maintaining criteria

Once a criteria set is locked, maintenance keeps the definition of success accurate without moving the goalposts after you've seen the results. Products release new versions, recommended patterns change, and scenarios evolve. Each can change what a valid result looks like.

Version the criteria set

When a product or SDK release changes correct usage, or when the scenario or starting workspace changes materially, review the criteria. If the changes affect what passes, fails, or skips, create a new version of the criteria set. Keep previous results tied to the version that produced them.

Put every revised criteria set through the same calibration process before locking the new version. Calibration is part of writing the revision; versioning and preserving its relationship to previous results are the maintenance work around it.

Results produced by different criteria-set versions don't form a clean trend. You can compare them for investigation, but don't present the difference as AX lift or regression without accounting for the changed yardstick.

Keep criteria for older product versions while those versions remain supported. A scenario targeting an older supported release should be evaluated against the guidance for that release, not today's latest recommendation. Retire the criteria when the product version is no longer supported.

If you discover a defect in a locked criterion after a production evaluation, document it with the affected results and correct it in a new criteria-set version. Keep the original results with a clear warning rather than silently changing their meaning so future analysis avoids comparing apples to oranges.

Watch the system around the criteria

The judge can evolve too. A changed judge model or prompt shouldn't silently change verdicts for a locked criteria set. The evaluation system needs to preserve backward compatibility, validate the change against existing evaluations, or restore affected results. If your evaluation system can't make that guarantee, treat a judge change as a new measurement boundary and don't compare its scores directly with earlier results.

Maintaining criteria takes two kinds of ownership, whether they sit with one person or two teams: someone needs to own whether the criterion describes correct product usage and someone needs to own whether the wording can be judged consistently. Drop either and the criteria set stops being trustworthy.

Common traps

Letting an LLM write your criteria. Hand the model your scenarios to generate the criteria and it fails three ways: its knowledge cutoff bakes stale patterns into what you're measuring, it can't know what you actually care about, and what it writes is either biased (when it also judges) or too vague for a different judge to score consistently.

Instead, draft them yourself, then use a model to tighten the wording and catch ambiguity.

Testing for style instead of correctness. "Code is well-structured and follows best practices" is not an AX criterion. Focus on whether the agent used your technology correctly. And where "best practices" matter, define them instead of letting the LLM infer the answer.

Testing for things the agent can't control. If the harness decides which tools to load, don't write a criterion that penalizes the agent for not loading a tool.

Conflating multiple concerns in one criterion. "Uses correct SDK with proper authentication and appropriate error handling" is three criteria pretending to be one. Split them so you get clear signal on each. This is another place to use LLMs to help refine your criteria after initial drafting.

Using vague language the judge can't verify. "Correct," "proper," "current," "appropriate," "best practices." If you wouldn't accept these words from a junior dev in a code review, don't use them in criteria.

Your turn

Write 3 criteria for your scenario. Before you move on, ask yourself:

- Would a perfect implementation pass?
- Would a clearly broken implementation fail?
- Would the judge produce the same verdict if you ran it five times on the same output?

If you're not confident about that last one, your criteria are ambiguous. Tighten the language until there's nothing left to interpret. The quality checklist in Appendix A has the full self-review process.

Then calibrate the criteria against known artifacts, get accuracy and consistency confirmed, and lock the criteria-set version before running the production evaluation.

Further reading

- [Building AX evals that actually work](#). More on the anatomy of a reliable criterion, balancing skip conditions, and calibrating the judge.

Chapter 6. Choosing agent profiles

Every combination of harness, model, tools, and a myriad of other configuration levers shapes and impacts the outputs of an AX evaluation. It is easy to get overwhelmed by the full matrix of possible combinations, so instead define **profiles** for testing.

An agent profile is a combination of the **operating system**, the **harness** (eg. GitHub Copilot in Visual Studio Code, Cursor, Claude Code CLI), the **model** and its **settings** (reasoning level and context window size), and the **extensions** (skills, MCP servers, instruction files). And, naturally, the versions of all the above.

Each profile is an environment the agent runs in, chosen to test a hypothesis, such as “adding skill X improves results at a lower cost.”

Start with the baseline

Always run a bare profile first: harness + model, no extensions. This tells you what the agent does with your technology out of the box: its training data plus the harness's built-in tools, such as web search.

The baseline is your reference point. If the baseline is already strong, your extensions need to add value without creating drag (i.e. unnecessary token cost or time). If the baseline is weak, you know where the gaps are and your extensions need to fill them. And if there's no extension yet, there's nothing to compare against, so running the baseline alone is perfectly fine. It still tells you whether there are gaps and helps you decide how to address them.

Add extensions one at a time

After the baseline, add agent-facing extensions incrementally. Test each skill, MCP server, plugin, or instruction set in isolation before testing combinations. Each combination tests a different question:

- **Bare:** What does the model already know about your technology?
- **One extension:** Does this extension add lift or drag?
- **Another extension:** Does this extension help independently?
- **Full stack:** Do the extensions compose well or interfere?

We've seen extensions improve outcomes independently but perform worse when combined (due to conflicting tool names, crowded context window, etc). It turns out, that composition isn't additive. You can't assume that two good extensions produce a better result than either one alone.

Common profiles and what they test

Profile	What it tests
Bare (no extensions)	Training data coverage for your technology
Skill only	Whether the skill adds lift or drag
Skill + docs MCP server	Whether general docs help or hurt alongside the specific skill
Full extension stack	Real-world developer experience (most realistic, hardest to diagnose)
Different harness, same model	Whether your extensions work across harnesses

For most teams starting out, **bare + one extension** is enough. That gives you a baseline and a delta.

When custom profiles matter

If you count the possible permutations of operating systems, harnesses, models, reasoning levels, and extensions, not to mention scenarios, you end up with an endless list of options. Enough to paralyze you before you even start.

Keep it pragmatic: for the baseline, use the profile that represents most of your audience. Which OS do they use? Which agent and model do they typically work with? Start there, then look at other combinations. With limited time, focus first on where you can improve the experience for the most developers.

If developers using your technology commonly have specific MCP servers installed (like a cloud provider's MCP server), include those in your profiles. Your extensions don't exist in isolation - they compete with whatever else is in the developer's context window.

Don't add custom profiles for the sake of coverage. Each profile multiplies your run count and makes interpretation harder. Add a profile when you have a specific question it answers.

Further reading

- [When the model has never seen your code](#). What the baseline tells you when your technology is proprietary or internal and the model has nothing to start from.
- [Your work might not need the smartest model](#). Set the bar before you run the eval, then find the least expensive model that clears it.

Chapter 7. Running evals and generating the readout

This is the mechanical part. The hard thinking is behind you. Now you execute and let the data come back.

Choosing an evaluation system

The methodology in this playbook doesn't depend on a specific evaluation system. You can use one your team built, an off-the-shelf evaluation product, or [Scope](#), the open-source evaluation system we built in DevRel at Microsoft. What matters is whether the system can run a representative evaluation and produce evidence you can trust.

That comes down to a handful of capabilities. The system needs to reproduce the environment your developers use, because a convenient setup like a CLI-based agent on Linux-based cloud runners tells you little when most of your audience works in an IDE on Windows. It needs to reset every run to the same starting workspace and run each scenario-profile combination repeatedly.

It needs deterministic checks for what's mechanically verifiable, such as build gates or structured checks against package.json, and an LLM judge with explicit pass, fail, and skip conditions for decisions that depend on meaning. Text matching can't tell "used Platform X" from "didn't use Platform X." Finally, it needs to keep the evidence behind every verdict (artifacts, execution results, costs, and full trajectories) even after results are aggregated. The full list is in Appendix A under "Evaluation system readiness."

If your current system covers these, keep using it. If the gaps are small, extending it might be the most pragmatic choice. If reproducing representative environments or producing trustworthy verdicts would take substantial custom work, adopting an existing system might cost less than building the missing pieces yourself.

Where Scope fits

This playbook isn't just a method you're left to build tooling for. Scope implements it: you configure scenarios, criteria, and agent profiles, run them repeatedly, and get a readout backed by the evidence from every run. It's open source, so you can use it as-is, extend it, or use it as a reference for your own system. You don't need Scope to use this playbook, but you don't have to start from scratch either.

Running the evaluation

Use the evaluation system available to you to configure the scenario, criteria, starting workspace, and agent profiles. Then run each scenario-profile combination multiple times (at least 5) because agent behavior varies between runs. A single run can be an outlier. Multiple runs tell you whether a result is signal or noise.

For every run, preserve the generated output, criteria verdicts, execution gate results, token or cost data, and the agent trajectory. Without the trajectory, you can see what failed but not reliably diagnose why.

Generating the readout

When runs are done, aggregate the raw results into a readout. Whether your evaluation system generates it automatically or you build it separately, the readout should combine criteria verdicts, gate outcomes, task costs, and profile comparisons into a report you can act on.

The readout translates "I ran some evals" into "here's what agents do with this technology, here's what's broken, and here's what to fix." Done right, they help find the needles in the haystack of collected data.

Further reading

- [The hidden variables in your agent eval](#). How the operating system, file paths, user identity, and tooling quietly change results between machines, and how to control for them.
- [Your AI coding agent evaluation is only as good as its sandbox](#). Why a correct answer can still invalidate the measurement, and how to define the sandbox around what the agent is allowed to know.
- [How to test agent skills without hitting real APIs](#). Evaluate skills that call APIs against lightweight emulation instead of production endpoints.

Part 3: From results to impact

Chapter 8. Interpreting the readout

The readout translates agent behavior into actionable information: what works, what doesn't, and what to fix.

A useful readout combines the results with interpretation and recommended next steps. Whether it is generated by your evaluation system or assembled separately, you need to understand what you're reading so you can explain it, defend it, and act on it.

What a readout should contain

A readout covers each scenario-profile combination. Whatever system produces it, here's what it should include:

First, the **prerequisite gate**: did the agent pick the right technology? If you asked for Cosmos DB and the agent built on MongoDB, everything else is irrelevant. The prerequisite gate must pass before the other gates and criteria are evaluated.

The **output evaluation** tells you whether the agent's code builds, runs, and deploys. These are mechanical gates, pass or fail. If the code doesn't build, nothing else matters for that run.

The **criteria results** show how the agent performed on each criterion across all runs. A criterion that passes 5/5 is strong. One that passes 2/5 is inconsistent. One that passes 0/5 is a clear gap.

The **profile comparison** is where you see lift or drag. If the bare profile scores 4/5 on a criterion and your skill profile scores 2/5, your skill is making things worse.

Task cost shows what each profile costs to run. Tokens come at different prices (input, output, cache tokens each have different price tiers), and what matters is the total cost per task, not just the token count. Drag at a higher price is easy to spot. Lift at a 3x the price

leads to an interesting discussion: is it worth it? Is there a way to optimize?

Finally, **recommended next steps** suggest what to investigate or fix. Whether your system generates them or you write them yourself, treat them as starting points, not conclusions. Your domain expertise determines which ones to pursue.

What to look for first

- **Is your extension even being used?** Just because you added an extension doesn't mean the agent calls it. Check the readout, and if needed the trajectories, to see if the extension was invoked. If not, it's dead weight costing users money (and wasting your time building and maintaining something agents aren't using).
- **Criteria where the bare profile outperforms an extension profile.** This is active drag. Investigate why.
- **Criteria that fail consistently (0/5) across all profiles.** A gap no extension is filling.
- **Criteria that are inconsistent (2/5, 3/5) across runs.** The agent is on the edge of a decision. Small content changes can tip inconsistent results toward consistent passes.
- **Large cost differences between profiles for similar outcomes.** Your extensions are adding expense without adding value.

What the readout can't tell you

A readout summarizes what happened, and it might include some explanation of why, but it's aggregated across multiple runs and profiles. When you need to understand exactly what path the agent followed in a specific run, you'll look at the collected trajectories. That's what the next chapter is for.

Further reading

- [Not all model upgrades are upgrades](#). How cheaper tokens led to higher bills and worse output, and why every model upgrade is a hypothesis to measure.

Chapter 9. Investigating agent trajectories

This is where you develop intuition. Reading trajectories is like watching an agent think out loud about your technology and discovering what it actually believes vs. what you intended. If the readout tells you what failed, the trajectories tell you why.

A trajectory is the complete record of what the agent did during a single run: every tool call, every file read, every decision. [ATIF \(Agent Trajectory Interchange Format\)](#) is one format for storing this data and comparing it across the different agents. When a criterion fails, the trajectory shows you the moment things went sideways.

Use whichever trajectory viewer your evaluation system provides. For ATIF files, we like to use the [ATIF Preview VS Code extension](#), which renders tool calls, file operations, and decision points in a readable format, allowing you to quickly find the relevant information.

How to read a trajectory

A trajectory is a sequence of events. Each event records an action: reading a file, calling a tool, generating code, receiving tool output. You're looking for decision points, the moments where the agent chose one path over another.

Some models also expose their reasoning (chain-of-thought). If present, this is great additional information to understand *why* the agent chose to do something. Look at the reasoning to see what the agent considered and what it dismissed.

Scan for:

- **Tool calls.** Which tools did the agent call? Which were available but never called? If your extension was loaded but never invoked, the agent decided it didn't need your help.
- **File reads.** Which docs did the agent read? Did it find the right page? Did it read the whole page or stop partway?

- **Decision sequences.** Follow the chain of events leading to a wrong output. Often the agent made the wrong call early, and everything after was downstream of that first mistake.

Discovery, invocation, and application

When something goes wrong, the first question is: at what stage did the agent fail?

Discovery failure: The agent failed to load your extension entirely. The MCP server didn't start, or there was an error loading its tools, or the skill exceeded the agent's size limits. Your extension never made it into the context. This is an infrastructure problem.

Lack of invocation: The extension loaded successfully, you can see it in the trajectory, but the agent decided not to use it. This typically means either the tool's description doesn't match the intent the model inferred from the user's instruction, or the model was confident enough in its own knowledge and didn't see a need for additional information.

Application failure: The agent found your technology and tried to use it but applied it incorrectly. It chose the wrong SDK version, wrong pattern, or ignored critical configuration. This is all a content problem. The fix is usually in your docs, skill content, or tool responses.

Why do these distinctions matter? Because the fixes are completely different. A discovery failure is an infrastructure fix. Lack of invocation needs better tool descriptions or vocabulary alignment. An application failure needs better content or plan-invalidation language in your docs.

Contradictory behaviors

One of the most common findings: the agent does X in one run and the opposite in another, given the exact same scenario and profile. For example, Run 1 uses your CLI tool. Run 2 ignores it and edits files manually. Same prompt, same extensions, but different behavior.

This is how LLMs work. The model's token sampling introduces variability. When the agent is on the boundary of a decision, small sampling differences tip it one way or the other.

Contradictory behavior tells you the agent's decision is weakly motivated. Something in the context is almost-but-not-quite convincing enough to consistently steer behavior. And that's actually good news: these are the highest-value improvements. A small content change can tip a 2/5 result to 5/5 because the agent was already close to the right decision.

Attribution: whose problem is it

When you find a failure, trace it to its source:

- **The documentation's problem.** Ambiguous content, misleading emphasis, or missing information.
- **The extension's problem.** Your skill or MCP tool provided content that confused the agent or conflicted with other sources.
- **The training data's problem.** The model's training data is stale and no extension is correcting it. The most common root cause.
- **The agent's problem.** The agent is the harness and the model together. A reasoning error, or harness behavior like a fetch tool that pulls in irrelevant page content, that no content change would fix. Rare. When it's the harness, report it to the harness vendor.

The source determines the fix. A doc problem needs a doc fix. A training data problem needs an extension that provides current information. **Don't band-aid a doc problem with a skill when you could fix the doc.** We can't emphasize this enough.

If you own the source, fix it directly. If you don't, bring the evidence to the team that does. A new skill should be the last resort because the improvement then depends on developers finding, installing, and updating it. Everyone without the skill still has the broken experience.

Investigating: a real example

In an SPFx upgrade evaluation targeting v1.22.0, the bare profile installed the wrong version in 4 out of 5 runs. The readout showed only the symptom: a failed version criterion. The trajectory showed the cause. The agent fetched the migration guide, read npm commands hardcoded to v1.22.1, and followed them instead of the version the user asked for.

That moved the investigation away from an abstract model weakness and toward a specific documentation fix. Appendix B follows the complete evaluation, from scenario design through the shipped fixes.

Further reading

- [How AI coding agents actually use your technology](#). A step-by-step walk from prompt to generated code. It's a map of what you're reading in a trajectory.

Chapter 10. Common failure patterns

After spending the better part of two years living in coding agent evaluations, we've identified patterns repeat across every technology we've evaluated. Learning them means you won't start from scratch. You'll recognize what you're seeing and know where to look next. Consider them "jump starts" on your own agent evaluation intuition.

Each pattern describes what you'll see, gives a real example, and points to what to fix.

1. The agent ignores your extension and uses training data instead

Your extension is loaded: you can see it in the trajectory. But the agent never calls it. Instead, it generates code from training data, which is outdated or wrong.

The model feels confident enough from what it learned in training and doesn't see a reason to invoke your tool. Often the issue is vocabulary: your tool says "manage database operations" but the developer asked to "build an API." The agent doesn't connect the two.

Rewrite tool descriptions to match how developers and models think about the task. Include vocabulary for all task types: "build," "create," "diagnose," "compare." And measure baseline first. If the model already does well without your tool, the tool might not be needed for this scenario.

2. The agent follows docs too literally

This one's subtle. The agent finds your docs, reads them, and follows them. That sounds like success, right? Except the docs include `npm install @azure/cosmos@4.1.0` as an example, and the agent

installs 4.1.0 even though the developer asked for the latest version. It anchored on the version number in the example.

Agents treat retrieved content as authoritative. Whatever version, pattern, or approach your docs foreground becomes what the agent does. If your docs cover multiple approaches but emphasize one, the agent picks the emphasized one and ignores the rest.

Use version variables or `@latest` in doc examples instead of hardcoded versions. Ensure balanced coverage when a page describes multiple approaches. If one approach is recommended, say so explicitly rather than relying on emphasis.

Also: keep your docs up to date. When agents retrieve docs while working on a task (as opposed to relying on crawled training data), up-to-date docs are the easiest and cheapest way to fix agent behavior.

3. The agent has a plan before it reads your content

The agent already *knows* how to do the task. It reads your docs but treats them as confirmation, not instruction. You added a tip pointing to the correct CLI command. The agent ignored it in every run. Why? Because tips are suggestions, and the agent had already decided what to do.

LLMs form plans from training data before consuming extension content. Suggestions sit alongside the existing plan as one more input. Only explicit contradiction forces plan abandonment.

Don't describe the right path. Invalidate the wrong one. "Manually updating package.json alone will result in build failures" works. "Consider using the CLI instead" doesn't. Use this sparingly, though. If you put plan-breaking language everywhere, it becomes noise.

4. The agent picks the wrong tool when multiple overlap

Multiple extensions could handle the same task. The agent picks the wrong one, calls both and mixes patterns from each, or calls one and assumes it covered everything.

The issue is vocabulary overlap. “Manage database operations” vs. “database management and queries.” The agent can’t distinguish which matches the developer’s intent, and its choice depends on factors you don’t control: context position, training data associations, or just which tool appears first.

Use distinctive, product-specific vocabulary. “Configure Cosmos DB hierarchical partition keys” instead of “manage database settings.” Test alongside the most common extensions developers in your space use.

5. Too many extensions make everything worse

The full extension stack produces worse results than the bare baseline. You add more tools and the scores go *down*.

Every extension consumes context tokens whether it's used or not. Past a threshold, the cumulative cost exceeds the benefit. We measured this directly: loading several overlapping extensions for the same platform at once used 4x the tokens while producing the worst results across 2 out of 3 scenarios. The agent read too many tool descriptions, got conflicting guidance, and made worse decisions than it would have with training data alone. None of those extensions was the problem on its own. The combination was.

Track net lift across the full stack. If adding an extension doesn’t measurably improve outcomes, remove it. “Install everything and let the agent figure it out” is a losing strategy.

Communicate this in your documentation too, so that developers working with your extensions are aware of it. If your extension works best as a standalone, say so.

6. The agent does manual work instead of running your CLI

Your CLI provides a ready-to-execute command. But the agent ignores it and implements each step manually by editing files. The manual edits introduce version errors the CLI would have prevented.

Why would it do that? Because the agent’s default behavior is to use file editing tools. That’s its primary modality. CLI commands in docs or tool responses are treated as specifications to implement, not commands to run.

Foreground the script as the primary action. Add explicit instructions: “Run the provided commands rather than manually editing files.” Consider output formats that make scripts unambiguous actions rather than readable documents.

7. The agent anchors on retrieved content over correct knowledge

This is the counter-intuitive one. The agent’s training data has the right answer. But after calling a tool or reading a page, it overwrites its correct knowledge with whatever the retrieved content emphasizes. The result is *worse* than baseline.

Models give disproportionate weight to retrieved content. If your docs page emphasizes one architectural pattern, the agent anchors on that pattern and forgets the other two it actually knew about. We saw this in one of our tests: the bare baseline correctly distinguished three multi-tenancy models. Adding a docs skill that routed to a page emphasizing one model caused the agent to collapse all three into one.

Ensure retrieved content provides balanced coverage. If your tool returns docs, make sure those docs don't inadvertently steer the agent away from patterns it already knows.

8. The agent scaffolds from 2020

The agent uses an outdated version of your technology. An old scaffolding tool, a deprecated SDK, a legacy API pattern. And it does this *confidently*, as if the old version is still current.

Training data volume is the culprit. Your old tool has years of blog posts, tutorials, and Stack Overflow answers. Your new tool has months of content. The model's confidence is proportional to how much it has seen, not how recent it is. Runtime resolution can compound this too: npx without a pinned version can silently resolve to a package published in 2020.

Put explicit deprecation signals in the sources you control: package deprecation metadata (like `npm deprecate`), a warning at the top of the old tool's README and docs, runtime warnings in the CLI, and redirects from outdated docs pages. "DEPRECATED: Do not use X. Use Y instead." works because it invalidates the plan the agent already has. Reach for an agent extension only to bridge the gap while those changes ship. Choose distinctive names for new tools rather than generic category descriptions.

9. The agent can't follow your tool's routing instructions

Your MCP server uses subrouting: a parent tool returns instructions telling the agent to call a subtool for the actual content. The agent treats the routing response as a final answer and never calls the subtool.

This happens when a competing tool in the same context returns actual content in one turn. The model decides it has enough information and stops. It optimizes for efficiency, and a direct answer beats a routing chain.

Test tools in realistic multi-extension compositions, not in isolation. If your subrouting pattern breaks when other tools are present, consider returning content directly instead of routing.

From failure to surface

The pattern tells you where to investigate first. It doesn't prove the fix, but it narrows the search.

What you observe	First surface to inspect
The extension never loads	Packaging, registration, startup, and harness limits
The extension loads but is never invoked	Tool or skill name, description, and vocabulary
The extension is invoked but the output is wrong	Returned content, response shape, docs, and instructions
The agent edits files instead of running the CLI	Docs examples, tool response, and CLI output format
The full extension stack performs worse than baseline	Overlapping descriptions, conflicting guidance, and context cost
The agent confidently uses an old version	Docs versions, deprecation signals, and runtime version resolution

Change one surface at a time and rerun the same evaluation. Otherwise, you won't know which change caused the result.

Further reading

- [Stop overloading your skills](#). Measure the baseline first, and put in your skill only what the model doesn't already know.
- [Your agent already has a plan](#). The full story behind invalidating the wrong path instead of describing the right one.

- [How to diagnose conflicts between AI agent extensions.](#)
Vocabulary collisions, guidance conflicts, and competition for context, and how to isolate the cause.
- [Stop skillmaxxing, save your tokens.](#) What dozens of installed skills cost before the developer types a single prompt.
- [Don't rewrite your CLI for agents.](#) Test data on flat arguments versus a single JSON payload, and why to measure before redesigning your CLI for agents.
- [Why AI coding agents keep using deprecated CLIs and SDKs.](#)
Training data gravity, and what to do when agents keep reaching for the tool you retired.
- [Your agent just scaffolded a project from 2020.](#) How npx and Node version managers silently resolve to old versions, and how to pin them.

Chapter 11. Turning findings into improvements

An evaluation has no impact until someone changes the source of the behavior. The owning team is whoever controls the affected surface: the documentation, SDK, API, CLI, extension, or other source the agent relies on.

How you turn evidence into a change depends on your relationship to that owner. You might own the surface yourself, collaborate directly with the people who do, or contribute through public channels. In every case, the goal is the same: fix the source, verify the result, and avoid adding another layer unless the source can't be changed.

If you own the affected surface

Start with failures that prevent the agent from selecting or using the technology at all. Then look for active drag, where an extension performs worse than the bare baseline. Inconsistent results deserve attention too. A 2 out of 5 or 3 out of 5 result often means the agent is close to making the right decision, and a small source change might tip it.

Trace each result to a surface your team controls. Fix unclear guidance in the docs. Change the name and description of a tool that isn't invoked. Clarify ambiguous output in the response shape or CLI format. Building another extension on top of a source problem adds another dependency without repairing the experience for everyone else.

Treat the proposed change as a hypothesis. Change one thing, rerun the same scenarios and profiles, and compare the result with the original baseline. If it doesn't improve the outcome, don't ship it just because the new wording sounds better.

If you can collaborate directly with the owner

Approach the work as a partnership, not a verdict on the quality of the team's product. Before you meet, understand what outcomes matter to them, what they already know about AX, and which agent-facing surfaces they maintain.

Lead with baseline results from a scenario relevant to their technology. No extensions, just: "Here's what happens when a developer asks a coding agent to build something with your SDK."

People who work on a technology every day know its vocabulary and intended patterns so well that those assumptions become invisible. Showing how an agent actually approaches the technology reveals where that internal perspective differs from the developer experience.

The owner might respond in several ways:

They want to test a fix. If the change is small, test it together. If the problem is broader, agree on additional scenarios before recommending a solution.

They expect models to improve. Push back on waiting: it leaves today's experience unchanged, with no guarantee the next model will behave differently. Simulate the smallest meaningful change the team could ship, for example by intercepting the agent's requests and serving the modified content, and show whether it makes results better, more consistent, or cheaper. A result the team can see before committing anything is far easier to say yes to.

They already measure AX. Compare scenarios, criteria, environments, and results. Where findings disagree, inspect the trajectories together. Different evaluation approaches can reveal different parts of the same experience.

They can't prioritize the change. Record the evidence and the constraint. Revisit it when priorities, contribution options, or the underlying technology change.

Match your involvement to what helps the owner act. They may ask you to collaborate on the evaluation, hand over the evidence, contribute a fix, or step back.

If you need to contribute through public channels

You won't always have a direct relationship with the owning team. Use the contribution path the project provides:

- Open an issue with the scenario, profile, criteria, run count, results, and a minimal reproduction.
- Link to the relevant outputs and trajectories, removing credentials, personal data, and other sensitive information first.
- Propose the smallest source-level change supported by the evidence.
- Submit a focused pull request when the project accepts external contributions.
- Use the project's private reporting or support channel when the evidence shouldn't be disclosed publicly.

Make it easy for maintainers to reproduce the result. Separate observations from conclusions, state the evaluation's limits, and include the baseline so they can see whether the affected surface helps or hurts.

The owner may disagree, lack capacity, or have constraints you can't see. That doesn't invalidate the finding, and it doesn't make the owner an obstacle. Document the evidence, keep the evaluation reproducible, and revisit it when something changes.

When the source can't change

Sometimes the source can't be fixed quickly because of priorities, release constraints, or a closed contribution process. A temporary skill or instruction set can bridge the gap until the owner ships a source-level fix.

Coordinate with the owning team when possible. If you publish an independent bridge:

- Label it clearly as independent and don't imply endorsement.
- State which gap it addresses and link to the supporting evidence.
- Name who maintains it and how updates will be handled.
- Define the condition for retiring it when the source is fixed.

A bridge creates another surface that users must find, install, trust, and update. Treat it as temporary infrastructure, not a substitute for repairing the source.

Presenting the evidence

Frame findings constructively. If a skill improves implementation tasks but not advisory scenarios, say exactly that. A bounded observation is more useful than a broad judgment about the quality of the product or the team's work.

Lead with what agents do, not with the evaluation machinery. Keep the scenarios, criteria, outputs, trajectories, and environment details available so others can inspect how you reached the conclusion.

Be precise about scope. Results belong to the tested scenario, profiles, versions, and point in time. They don't establish that a product, extension, or model is universally good or bad.

Keep measuring

AX improvement isn't one-off work. Coding agents, models, harnesses, extensions, and products change independently. What works today can regress even when the owning team hasn't changed anything.

Rerun the evaluation regularly using the same locked definition so results stay comparable. Set the cadence to match how fast the agents your audience uses are changing and how much capacity there is to investigate and act on what you find. A system like Scope can automate that repetitive work and keep the signal current. When the product, scenario, or criteria change, create a new version instead of presenting the results as a continuation of the old trend.

Following up

- If a fix shipped, rerun the evaluation and verify the result.
- If the owner requested more scenarios, run them and share the evidence.
- If no change was possible, record why and revisit in 2-3 months or when the product, model, or contribution path changes.
- If you published a bridge, check whether its retirement condition has been met.

Whether you own the surface, collaborate with its owner, or contribute through public channels, the loop is the same: measure, diagnose, change, and verify. The improvements compound the same way the problems did.

Further reading

- [How to test agent experience changes without shipping them](#). Why most hypotheses fail, and how to emulate a change before you open a pull request.

Appendices

Appendix A. Quality checklist

Use this checklist to self-review before running an evaluation or asking someone else to review its quality.

Scenario quality

- ☐ The prompt is something a developer would actually type
- ☐ Contains no evaluation metadata, scoring hints, or instructions about how to be evaluated
- ☐ Be mindful of vocabulary: developers may not know the internal product names or specific terms your team uses. Don't assume they'll use the "right" words

Criteria quality: accuracy

- ☐ You have a prerequisite gate: one or at most two unconditional checks. If they don't pass, further evaluation makes no sense. Example: you ask the agent to build a database solution assuming Cosmos DB (without naming it), and the agent writes MongoDB code. Evaluating the correctness of that MongoDB solution is pointless because the basic technology choice is wrong.
- ☐ Each criterion tests an aspect of your technology that matters for correct usage
- ☐ Pass conditions describe what correct usage looks like with specific packages, versions, or APIs named
- ☐ Fail conditions describe specific incorrect patterns (not just "does not use the technology correctly")
- ☐ Skip conditions cover cases where the criterion doesn't apply
- ☐ A perfect implementation would pass all criteria
- ☐ A clearly broken implementation would fail at least one criterion

Criteria quality: calibration

- ☐ Criteria are aligned with reasonable expectations for a developer using this technology, not with what your extension provides
- ☐ No criterion is so loose that everything passes
- ☐ No criterion is so strict that only one exact implementation passes
- ☐ Each criterion tests one thing (not multiple concerns in one criterion)
- ☐ Criteria test correctness, not style
- ☐ Criteria don't test things the agent can't control

Criteria quality: consistency

- ☐ Each criterion has explicit pass/fail/skip conditions that leave nothing to interpretation
- ☐ Criteria use technology-specific vocabulary the judge can verify (package names, API patterns, config keys) rather than vague quality terms

Criteria calibration

- ☐ Each criterion passes a known-good artifact and fails a known-bad artifact
- ☐ Repeated judgments of the same artifacts produce consistent verdicts
- ☐ Someone who knows the product has confirmed technical accuracy
- ☐ Repeated scoring has confirmed the wording produces consistent verdicts
- ☐ Calibration runs are kept separate from production results

Criteria maintenance

- ☐ The locked criteria set has a version, and production results record that version
- ☐ Material changes to the product, scenario, or starting workspace trigger a criteria review
- ☐ Criteria for older product versions are retained only while those versions remain supported
- ☐ Defects found after production are documented and corrected in a new criteria-set version
- ☐ Judge changes preserve backward compatibility or establish a clearly labeled measurement boundary

Profile selection

- ☐ Bare baseline is included (unless you're only comparing extension variations)
- ☐ If there's an extension to compare, at least one extension profile is included
- ☐ If there's no extension yet, running just the baseline is fine. It tells you whether gaps exist and helps you decide how to address them.
- ☐ Each profile tests a specific question

Before running

- ☐ At least 5 runs per scenario-profile combination
- ☐ Use environment and agent where the majority of your audience is to ensure you're optimizing for the maximum impact first

Ongoing measurement

- ☐ Rerun cadence reflects changes to the coding agents your audience uses and the team's capacity to respond
- ☐ Recurring evaluations use the same locked definition so that results remain comparable
- ☐ Product, scenario, or criteria changes start a new version rather than continuing the old trend

Evaluation system readiness

- ☐ Reproduces the operating system, harness, model, and extensions used by the largest audience cohort
- ☐ Configures scenarios, starting workspaces, criteria, and profiles independently
- ☐ Resets every run to the same starting workspace and runs each scenario-profile combination repeatedly
- ☐ Supports mechanical gates and structured checks for verifiable outcomes
- ☐ Supports context-aware criteria with explicit pass, fail, and skip conditions for semantic decisions
- ☐ Preserves generated artifacts, verdict evidence, execution results, task costs, and complete trajectories per run
- ☐ Compares extension profiles against a bare baseline, including quality and task cost
- ☐ Keeps run-level evidence available when results are aggregated

Appendix B. Worked example: SPFx end-to-end

A complete AX evaluation cycle for SharePoint Framework (SPFx) project upgrades, from scenario design through evaluation, investigation, and shipped improvements. This is a condensed version of the full engagement.

We wanted to understand how well GitHub Copilot handles SPFx project upgrades out of the box. We chose upgrades because they're one of the most impactful scenarios for SPFx developers: upgrading is how developers get the latest capabilities. The evaluation was a collaboration: the SPFx product team knew what a correct upgrade should look like, and we, as developer advocates, knew how to measure whether an agent gets there. That split is common, but it isn't required. The steps are the same whether you own the technology, advocate for it, or contribute from the outside, as Chapter 11 describes. What changes is who ships the fixes.

The scenario

Prompt: "Upgrade the project to 1.22.2"

No additional context, no guidance. Just the outcome a developer would type. The starting workspace contained an SPFx v1.21.1 web part project. We deliberately kept the prompt minimal to test how well the agent reasons about the project on its own.

The environment

We matched a typical SPFx developer setup:

- VS Code with GitHub Copilot Chat
- Windows 11
- Claude Sonnet 4.6 (a popular model at the time)

The criteria

We grouped criteria into four classifications:

- **Prerequisite gate.** Was the project upgraded to SPFx 1.22.2? We check the version of `@microsoft/sp-webpart-base` in `package.json`. If this is wrong, everything else is noise.
- **Dependency currency.** Did the agent move the project to the correct dependency state? Exact versions for runtime/build dependencies, required pins/overrides, removal of legacy Gulp-era packages.
- **Configuration correctness.** Did the agent perform the migration mechanics correctly? Config files, scripts, `.yo-rc.json`, `.gitignore`, SASS config, Heft migration details.
- **Idiomatic use.** Did the agent align with current SPFx conventions? Expected modern lint rules and patterns.

Plus execution gates: Prerequisite (passes), Build (compiles), Test (tests pass), Run (starts successfully).

The profiles

We started with three profiles:

- **Bare baseline.** GitHub Copilot Chat + Claude Sonnet 4.6, no extensions. The default developer experience.
- **+ Anti-hallucination skill.** A skill the SPFx team had already created, instructing the agent to verify SPFx facts against authoritative documentation. We describe the full evaluation in [Behind SPFx Dev Skills: testing what agents know and fixing what they miss](#).
- **+ Anti-hallucination skill + context7 MCP.** Adding general documentation retrieval on top.

Baseline results

Every run passed all execution gates (the project compiled and ran). But the criteria told a different story:

Classification	Baseline	+ Skill	+ Skill + context7
Prerequisite	5/5	5/5	5/5
Idiomatic use	6/10	10/10	9/10
Dependency currency	34/50	38/50	39/50
Configuration correctness	30/80	37/80	41/80

The skill provided some lift, most clearly in idiomatic use, but still left the project only partially upgraded. Adding context7 on top made little difference, and context7 itself contributed nothing. In 3 of 5 runs, its tools never loaded into the agent's context. In the other 2, they were available but never invoked. Available tools are not always used.

What investigation revealed

The agent already knows SPFx. Before invoking any tools, the agent identified the workspace as an SPFx project from the file structure alone, formulated an upgrade plan, and knew the URL of the release notes page. It was going to Microsoft Learn, Microsoft's documentation site, to confirm versions, not to discover what to do.

The model knows docs URLs but struggles with SemVer. The agent fetched `release-1.22` when it needed `release-1.22.2`. After reading the page, it realized "v1.22.0 is not v1.22.2" but never attempted to fetch the version-specific page. It treated `release-1.22` as "close enough."

Documentation hardcodes led the agent astray. The migration guide hardcoded `@1.22.1` in all npm install examples. In 4 of 5 bare runs, the agent faithfully followed these examples, installing v1.22.1 instead of the requested v1.22.0 (in the v1.22.0 scenario variant).

Hypothesis-driven fixes

We used [Dev Proxy](#), an API simulator, to intercept the agent's requests to Microsoft Learn and return modified pages. That let us test documentation changes before committing them to live docs. Several hypotheses were wrong. Had we shipped those changes without testing, we would have degraded the experience.

Fix 1: Rename release notes pages to use full version. Replacing `release-1.22.md` with `release-1.22.0.md`, and updating the version number in the contents, made the agent recognize that v1.22.0 and v1.22.2 are different pages. In the April baseline, none of the 4 runs that read the release notes followed up with the 1.22.2 page. When we reran the scenario in September against the live docs, 4 of 5 runs did. Shipped as [PR #10855](#).

Fix 2: Add a warning to invalidate the agent's manual plan. A tip saying "you can also use CLI for Microsoft 365" didn't change the agent's plan. In the April baseline, no run used the CLI. In our simulations, a warning saying "manually updating package.json alone will result in build failures" got the agent to discover and use the CLI on its own, with no prompt engineering from the developer. Shipped as [PR #10921](#).

Fix 3: Rewrite the migration guide. The migration guide contained step-by-step npm commands and JSON snippets. In our simulations, those actionable code blocks pulled the agent back to manual steps, even with the warning in place. We rewrote the migration guide as a conceptual page (describing *what* changes, not *how* to apply them manually), directing developers to the CLI. Shipped in the same PR.

Once both changes were live, the September rerun showed the combined effect. Every bare run discovered and used the CLI without being told, and configuration correctness reached 83/85, up from 30/80 in April. The criteria set gained one configuration criterion between the two runs, so read this as a before-and-after pattern, not an exact delta.

CLI for Microsoft 365: the highest-value tool

[CLI for Microsoft 365](#) is an open source CLI whose `spfx project upgrade` command generates the version-specific changes an incremental upgrade needs. When we explicitly told the agent to use it, dependency currency and configuration correctness improved dramatically:

Classification	Baseline	+ CLI for Microsoft 365
Prerequisite	5/5	5/5
Idiomatic use	6/10	6/10
Dependency currency	34/50	49/50
Configuration correctness	30/80	75/80

Notice, that idiomatic use didn't move. The CLI encodes the upgrade mechanics, not the lint rules and patterns that make the code look current.

We then tested whether the CLI report format affected the outcome, using the v1.22.0 variant of the scenario:

Format	Idiomatic	Deps	Config	Avg tokens
Markdown (default)	8/10	42/50	72/85	888K
Markdown, summary first	9/10	50/50	85/85	1,380K
JSON	9/10	50/50	85/85	1,133K
JSONL	7/10	43/50	84/85	2,927K

JSON was the clear winner. It tied the summary-first Markdown report for the best quality while using about 18% fewer tokens. Of the two structured formats, it beat JSONL on every dimension at less than half the token cost. The structured format removes parsing ambiguity, and the `resolutionType` field tells the agent exactly how to apply each change.

Harness findings

Not every finding belongs to the technology you're evaluating. While investigating, we noticed an opportunity in how the harness retrieves docs. Microsoft Learn can return pages directly as Markdown, which gives agents cleaner content than converting the full HTML page. We shared this with the GitHub Copilot teams, who added `Accept: text/markdown` support to GitHub Copilot Chat and GitHub Copilot CLI. When harness behavior affects how agents use your technology, share it with the harness vendor. It's part of your agent experience too.

What shipped

- Release notes pages updated to full patch versions ([PR #10855](#))
- Warning callout and rewritten migration guide ([PR #10921](#))
- `Accept: text/markdown` support in GitHub Copilot Chat and GitHub Copilot CLI
- Terminal polling behavior reported to VS Code team
- JSON output format recommended for CLI for Microsoft 365

Two doc changes and one harness fix, and none of them needed a new extension. They fixed the sources the agent already relied on, so every developer benefits without installing anything.

The readouts behind this example

The tables and results in this appendix come from three readouts. Each folder contains the readout, the per-run results, and the trajectories:

- [SPFx upgrade from v1.21.1 to v1.22.2, April 2026](#). The baseline, skill, and CLI for Microsoft 365 profiles.
- [SPFx upgrade to v1.22.0, April 2026](#). The bare-profile version errors from Chapter 9 and the CLI report format comparison.
- [SPFx upgrade from v1.21.1 to v1.22.2, September 2026](#). The rerun against the live docs after the fixes shipped.

Each readout includes `readout.json`, which holds the complete data and is easiest to explore in VS Code with the [Scope Preview extension](#). The April readouts also include a `readout.md` that renders directly on GitHub.